

Advanced Algorithms: Homework 2 Solutions

Due on Jan. 31, 2024 at 2:29pm EST

Professor Dana Randall Spring 2024

As stated in the syllabus, unauthorized use
of previous semester course materials is
strictly prohibited in this course.

Exercise 1

(KT Chapter 13, Exercise 1) The 3-coloring problem is a decision problem, but we can word it as an optimization problem as follows. Suppose we are given a graph $G = (V, E)$ and we want to color each node with one of three colors, even if we aren't necessarily able to give different colors to every pair of adjacent nodes. Rather, we say that an edge (u, v) is *satisfied* if the colors assigned to u and v are different.

Consider a 3-coloring that maximizes the number of satisfied edges, and let c^* be the number of edges this satisfies. Give a polynomial-time algorithm that produces a 3-coloring that satisfies at least $\frac{2}{3}c^*$ edges. If you want, your algorithm can be randomized; in this cases the *expected* number of edges it satisfies should be at least $\frac{2}{3}c^*$.

Solution

Randomized Solution: Uniformly assign every vertex a color, independent of all other vertices. Any fixed edge has a $\frac{2}{3}$ chance of being satisfied.

For each edge e , we have the indicator variable I_e that indicates if that edge is properly colored. So, the total number of edges that are properly colored is $X = \sum_{e \in E} I_e$.

By linearity of expectation, $E[X] = \sum_{e \in E} E[I_e] = \sum_{e \in E} P(I_e = 1) = \sum_{e \in E} \frac{2}{3} = \frac{2}{3}|E|$.

Deterministic Solution: This is a derandomization of the above randomized algorithm. We greedily color vertices one by one, at each step choosing the color that is least present among its *already colored* neighbors. While running our algorithm, we call an edge *fixed* if both of its endpoints are already colored.

Consider any step of our algorithm, where we color vertex v . At this step, we fix all edges between v and its already colored neighbors. Since we choose the *least* present color among these neighbors, at most $\frac{1}{3}$ of the edges that we fix are made unsatisfied. Thus at least $\frac{2}{3}$ of the edges that we fix at any step are satisfied.

Since every edge gets fixed eventually, it follows that this process will satisfy $\geq \frac{2}{3}$ of all edges.

Note that our goal was to have expectation at least $\frac{2}{3}c^*$, not $\frac{2}{3}|E|$. Our algorithm satisfies this because $c^* \leq |E|$. This concept is often used in Approximation Algorithms - to show that an algorithm that achieves α of the optimal solution (which is unknown), we can show that it achieves α of something bigger than the optimum (which is known).

Exercise 2

(KT Chapter 13, Exercise 2) Consider a country in which 100,000 people vote in an election. There are only two candidates on the ballot: a Democratic candidate (denoted D) and a Republican candidate (denoted R). As it happens, this country is heavily Democratic, so 80,000 people go to the polls intending to vote for D and 20,000 go intending to vote for R .

However, the layout of the ballot is a little confusing, so each voter, independently and with probability $\frac{1}{100}$, votes for the wrong candidate — that is, the one that he or she did not intend to vote for.

Let X denote the random variable equal to the number of votes received by the Democratic candidate D , when the voting is conducted with this process of error. Determine the expected value of X , and give an explanation of your derivation of this value.

Solution

There are two types of people who will vote Democratic: Democrats who vote correctly, and Republicans who vote incorrectly.

- Each Democrat has a $\frac{99}{100}$ chance of voting correctly, so the expected number of votes in this category is $80000 \cdot \frac{99}{100} = 79200$ votes.
- Each Republican has a $\frac{1}{100}$ chance of voting incorrectly, so the expected number of votes in this category is $20000 \cdot \frac{1}{100} = 200$ votes.

Thus, we expect 79400 total votes for the Democrats.

Exercise 3

(*KT Chapter 13, Exercise 6*) One of the (many) hard problems that arises in genome mapping can be formulated in the following way. We are given a set of n *markers* $\{\mu_1, \mu_2, \dots, \mu_n\}$ — these are positions on a chromosome that we are trying to map — and our goal is to output a linear ordering of these markers. The output should be consistent with a set of k *constraints*, each specified by a triple $\{\mu_i, \mu_j, \mu_k\}$, requiring that μ_j lie between μ_i and μ_k somewhere in the total ordering that we produce. Note that this constraint does not specify which of μ_i or μ_k should come first in the ordering, only that μ_j should come between them. It also is not saying that they need to be adjacent to each other in this order.

Now it is not always possible to satisfy all constraints simultaneously, so we wish to produce an ordering that satisfies as many as possible. Unfortunately, deciding whether there is an ordering that satisfies at least k' of the k constraints is NP-complete. (You do not have to prove this!)

Give a constant $\alpha > 0$ (independent of n) and an algorithm with the following property. If it is possible to satisfy k^* of the constraints, then the algorithm produces an ordering of markers satisfying at least αk^* of the constraints. Your algorithm may be randomized; in this case it should produce an ordering for which the *expected* number of satisfied constraints is at least αk^* .

Solution

- Randomized Algorithm:

Pick a permutation of the n markers $\{\mu_1, \dots, \mu_n\}$ uniformly at random.

Note that for any three elements μ_i, μ_j, μ_k , exactly one of them must be in the middle of the other two. By symmetry, it is equally likely to be any of these three elements. Thus, the probability that a particular constraint $\{\mu_i, \mu_j, \mu_k\}$ is satisfied is $\frac{1}{3}$.

As in problem 1, the expected number of satisfied constraints is $\frac{1}{3}k$ by linearity of expectation. Since $k^* \leq k$, our algorithm meets the requirements for $\alpha = \frac{1}{3}$.

- Deterministic Algorithm: Unlike the algorithm in Problem 1, this randomized algorithm may not have a clean derandomization - at least I don't know one.